CSA 1737 – ARTIFICIAL INTELLIGENCE

Laboratory Record – Experiments 6 to 10

Ex. No. 6: List Methods (Add, Append, Extend and Delete)

AIM

To write a Python program to implement list methods: add (insert), append, extend and delete (remove, pop, del, clear).

ALGORITHM

Step 1: Start the program.

Step 2: Create a list with some initial elements and display it.

Step 3: Add an element at a given position using insert().

Step 4: Add a single element at the end using append().

Step 5: Add many elements at the end using extend().

Step 6: Delete an element by value using remove(), the last element using pop(), an element by index using del, and all elements using clear().

Step 7: Display the list after each operation.

Step 8: Stop the program.

CODE

```
# Experiment 6: List Methods (Add, Append, Extend & Delete)
my_list = [10, 20, 30]
print("Original list      :", my_list)

# ---- ADD (insert at a given position) ----
my_list.insert(1, 15)
print("After insert(1,15) :", my_list)

# ---- APPEND (add one item at the end) ----
my_list.append(40)
print("After append(40)   :", my_list)

# ---- EXTEND (add many items at the end) ----
my_list.extend([50, 60, 70])
print("After extend       :", my_list)

# ---- DELETE ----
my_list.remove(15)          # delete by value
print("After remove(15)   :", my_list)

popped = my_list.pop()      # delete last item
print("After pop()        :", my_list, "| popped item =", popped)

del my_list[0]              # delete by index
print("After del my_list[0]:", my_list)

my_list.clear()             # delete all items
print("After clear()      :", my_list)
```

OUTPUT

```
Original list      : [10, 20, 30]
After insert(1,15) : [10, 15, 20, 30]
After append(40)   : [10, 15, 20, 30, 40]
After extend       : [10, 15, 20, 30, 40, 50, 60, 70]
After remove(15)   : [10, 20, 30, 40, 50, 60, 70]
After pop()        : [10, 20, 30, 40, 50, 60] | popped item = 70
After del my_list[0]: [20, 30, 40, 50, 60]
After clear()      : []
```

RESULT

Thus the Python program to implement list methods (add, append, extend and delete) was written and executed successfully, and the output was verified.

Ex. No. 7: Different Set Operations

AIM

To write a Python program to illustrate different set operations.

ALGORITHM

Step 1: Start the program.

Step 2: Create two sets A and B.

Step 3: Perform union (|), intersection (&), difference (-) and symmetric difference (^) using operators.

Step 4: Perform the same operations using the methods union(), intersection(), difference() and symmetric_difference().

Step 5: Check subset, superset and disjoint relations using issubset(), issuperset() and isdisjoint().

Step 6: Add and remove an element using add() and remove().

Step 7: Display the results.

Step 8: Stop the program.

CODE

```
# Experiment 7: Different Set Operations
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
print("Set A :", A)
print("Set B :", B)

print("\nUnion (A | B)                    :", A | B)
print("Intersection (A & B)            :", A & B)
print("Difference (A - B)              :", A - B)
print("Difference (B - A)              :", B - A)
print("Symmetric Difference (A ^ B)    :", A ^ B)

print("\nUsing methods:")
print("A.union(B)                :", A.union(B))
print("A.intersection(B)         :", A.intersection(B))
print("A.difference(B)           :", A.difference(B))
print("A.symmetric_difference(B) :", A.symmetric_difference(B))

C = {1, 2}
print("\nSet C :", C)
print("C is subset of A   :", C.issubset(A))
print("A is superset of C :", A.issuperset(C))
print("A and B disjoint   :", A.isdisjoint(B))

A.add(10)
print("\nAfter A.add(10)     :", A)
A.remove(10)
print("After A.remove(10)  :", A)
```

OUTPUT

```
Set A : {1, 2, 3, 4, 5}
```

```
Set B : {4, 5, 6, 7, 8}

Union (A | B)                    : {1, 2, 3, 4, 5, 6, 7, 8}
Intersection (A & B)             : {4, 5}
Difference (A - B)               : {1, 2, 3}
Difference (B - A)               : {8, 6, 7}
Symmetric Difference (A ^ B)     : {1, 2, 3, 6, 7, 8}

Using methods:
A.union(B)                : {1, 2, 3, 4, 5, 6, 7, 8}
A.intersection(B)         : {4, 5}
A.difference(B)           : {1, 2, 3}
A.symmetric_difference(B) : {1, 2, 3, 6, 7, 8}

Set C : {1, 2}
C is subset of A   : True
A is superset of C : True
A and B disjoint   : False

After A.add(10)     : {1, 2, 3, 4, 5, 10}
After A.remove(10)  : {1, 2, 3, 4, 5}
```

RESULT

Thus the Python program to illustrate different set operations was written and executed successfully, and the output was verified.

Ex. No. 8: Calendar for the Given Month and Year

AIM

To write a Python program to generate the calendar for the given month and year.

ALGORITHM

Step 1: Start the program.

Step 2: Import the calendar module.

Step 3: Read the year and the month from the user.

Step 4: Call calendar.month(year, month) to generate the calendar text.

Step 5: Display the calendar.

Step 6: Stop the program.

CODE

```
# Experiment 8: Calendar for the given month and year
import calendar

year = int(input("Enter year : "))
month = int(input("Enter month (1-12) : "))

print()
print(calendar.month(year, month))
```

OUTPUT

```
Enter year : 2026
Enter month (1-12) : 10

    October 2026
Mo Tu We Th Fr Sa Su
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
```

RESULT

Thus the Python program to generate the calendar for the given month and year was written and executed successfully, and the output was verified.

Ex. No. 9: Remove Punctuations from the Given String

AIM

To write a Python program to remove punctuations from the given string.

ALGORITHM

Step 1: Start the program.

Step 2: Define a string containing all punctuation characters.

Step 3: Read a string from the user.

Step 4: Create an empty string called no_punct.

Step 5: For each character in the input string, add it to no_punct only if it is not a punctuation character.

Step 6: Display the string without punctuation.

Step 7: Stop the program.

CODE

```
# Experiment 9: Remove punctuations from a given string
punctuations = '''!()-[]{};:'"\\,<>./?@#$%^&*_~'''

my_str = input("Enter a string: ")

no_punct = ""
for char in my_str:
    if char not in punctuations:
        no_punct = no_punct + char

print("String without punctuation:", no_punct)
```

OUTPUT

```
Enter a string: Hello!!!, he said ---and went.
String without punctuation: Hello he said and went
```

RESULT

Thus the Python program to remove punctuations from a given string was written and executed successfully, and the output was verified.

Ex. No. 10: 8-Puzzle Problem

AIM

To write a Python program to solve the 8-Puzzle problem using the A* search algorithm.

ALGORITHM

Step 1: Start the program.

Step 2: Represent the puzzle as a tuple of 9 values where 0 is the blank tile; define the goal state (1 2 3 / 4 5 6 / 7 8 0).

Step 3: Define the heuristic h(n) as the sum of Manhattan distances of all tiles from their goal positions.

Step 4: Put the start state in a priority queue with f(n) = g(n) + h(n).

Step 5: Remove the state with the lowest f(n). If it is the goal, return the path.

Step 6: Otherwise generate its successors by moving the blank up, down, left and right, and insert the unvisited ones into the queue.

Step 7: Repeat steps 5 and 6 until the goal is reached.

Step 8: Display the sequence of states from start to goal.

Step 9: Stop the program.

CODE

```
# Experiment 10: 8-Puzzle problem using A* search (Manhattan distance)
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def manhattan(state):
    dist = 0
    for i, tile in enumerate(state):
        if tile != 0:
            g = GOAL.index(tile)
            dist += abs(i // 3 - g // 3) + abs(i % 3 - g % 3)
    return dist

def neighbours(state):
    result = []
    z = state.index(0)
    r, c = divmod(z, 3)
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            n = nr * 3 + nc
            s = list(state)
            s[z], s[n] = s[n], s[z]
            result.append(tuple(s))
    return result

def solve(start):
    heap = [(manhattan(start), 0, start, [start])]
    visited = set()
    while heap:
```

```
        f, g, state, path = heapq.heappop(heap)
        if state == GOAL:
            return path
        if state in visited:
            continue
        visited.add(state)
        for nxt in neighbours(state):
            if nxt not in visited:
                heapq.heappush(heap, (g + 1 + manhattan(nxt), g + 1, nxt, path +
[nxt]))
    return None

def show(state):
    for i in range(0, 9, 3):
        print(*[t if t != 0 else '_' for t in state[i:i + 3]])
    print()

start = (1, 2, 3, 0, 4, 6, 7, 5, 8)
path = solve(start)
print("Solved in", len(path) - 1, "moves\n")
for step, s in enumerate(path):
    print("Step", step)
    show(s)
```

OUTPUT

```
Solved in 3 moves

Step 0
1 2 3
_ 4 6
7 5 8

Step 1
1 2 3
4 _ 6
7 5 8

Step 2
1 2 3
4 5 6
7 _ 8

Step 3
1 2 3
4 5 6
7 8 _
```

RESULT

Thus the Python program to solve the 8-Puzzle problem using A* search was written and executed successfully, and the output was verified.